

# Pipelined Processors in Ripes

## Homework 2

### Part 1: Identifying hazards

#### Four-stage pipeline control hazards

|                       |   |    |     |     |     |     |     |     |     |    |
|-----------------------|---|----|-----|-----|-----|-----|-----|-----|-----|----|
| addi s0 x0 1          | I | EX | MEM | WB  |     |     |     |     |     |    |
| addi s1 x0 3          |   | I  | EX  | MEM | WB  |     |     |     |     |    |
| jal addReg            |   |    | I   | EX  | MEM | WB  |     |     |     |    |
| addi s0 x0 5          |   |    |     | I   | EX  | MEM | WB  |     |     |    |
| addi s1 x0 6          |   |    |     |     | I   | EX  | MEM | WB  |     |    |
| addReg : addi s2 x0 1 |   |    |     |     |     | I   | EX  | MEM | WB  |    |
| addi s3 x0 1          |   |    |     |     |     |     | I   | EX  | MEM | WB |



The PC reg multiplexer selects the newly computed address instead of fetching addi s3 x0 1 instruction.



The Ex/Mem and I/Ex registers need to be cleared, to remove in-flight instructions from the pipeline.

#### Six-stage pipeline control hazards

|                       |    |    |    |      |      |      |      |      |      |      |      |      |    |
|-----------------------|----|----|----|------|------|------|------|------|------|------|------|------|----|
| addi s0 x0 1          | IF | ID | EX | MEM1 | MEM2 | WB   |      |      |      |      |      |      |    |
| addi s1 x0 3          |    | IF | ID | EX   | MEM1 | MEM2 | WB   |      |      |      |      |      |    |
| jal addReg            |    |    | IF | ID   | EX   | MEM1 | MEM2 | WB   |      |      |      |      |    |
| addi s0 x0 5          |    |    |    | IF   | ID   | EX   | MEM1 | MEM2 | WB   |      |      |      |    |
| addi s1 x0 6          |    |    |    |      | IF   | ID   | EX   | MEM1 | MEM2 | WB   |      |      |    |
| addi s1 x0 7          |    |    |    |      |      | IF   | ID   | EX   | MEM1 | MEM2 | WB   |      |    |
| addReg : addi s2 x0 1 |    |    |    |      |      |      | IF   | ID   | EX   | MEM1 | MEM2 | WB   |    |
| addi s3 x0 1          |    |    |    |      |      |      |      | IF   | ID   | EX   | MEM1 | MEM2 | WB |



The PC reg multiplexer selects the newly computed address instead of fetching addi s3 x0 1 instruction. (The jump logic takes effect in the Mem1 stage of the processor)



The Ex/Mem1, Id/Ex and If/Id registers need to be cleared, to remove in-flight instructions from the pipeline.

### Four-stage pipeline data hazards

|              |   |    |     |     |     |     |     |     |     |     |     |    |
|--------------|---|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|----|
| lw s1 0(sp)  | I | EX | MEM | WB  |     |     |     |     |     |     |     |    |
| addi s2 s1 4 |   | I  | EX  | EX  | MEM | WB  |     |     |     |     |     |    |
| lui s0 1023  |   |    | I   | I   | EX  | MEM | WB  |     |     |     |     |    |
| addi s0 s0 5 |   |    |     | ... | I   | EX  | MEM | WB  |     |     |     |    |
| ori s2 s0 1  |   |    |     |     |     | I   | EX  | MEM | WB  |     |     |    |
| srli s3 s0 4 |   |    |     |     |     |     | I   | EX  | MEM | WB  |     |    |
| sw s0 8(sp)  |   |    |     |     |     |     |     | I   | EX  | MEM | WB  |    |
| lw s1 8(sp)  |   |    |     |     |     |     |     |     | I   | EX  | MEM | WB |

 s1 that is loaded at the Mem stage has to be used as an argument for the addi s2.

 We stall for 1 clock tick.

 s0 that is loaded needs to be forwarded to addi s0 s0 15 to have the correct input value to perform the addi on.

 s0 that is loaded needs to be forwarded to ori s2 s0 1 to have the correct input value to perform the ori on.

 We tiebreak s0 with the most recent result. It can be from the previous (ex/ex), the penultimate (mem/ex).

 No hazard with reg file read bypass. The write back of s0 of lui will have ended before the read of s0 of srli. *SUBCOMPONENT(registers, TYPE(RegisterFile<XLEN, true>));*

 No store load hazard because the mem stage of store will have ended before the mem stage of load would begin.

## Six-stage pipeline data hazards

|              |    |    |    |          |          |     |          |          |          |          |          |          |          |          |    |  |
|--------------|----|----|----|----------|----------|-----|----------|----------|----------|----------|----------|----------|----------|----------|----|--|
| lw s1 0(sp)  | IF | ID | EX | MEM<br>1 | MEM<br>2 | WB  |          |          |          |          |          |          |          |          |    |  |
| addi s2 s1 4 |    | IF | ID | ID       | ID       | EX  | MEM<br>1 | MEM<br>2 | WB       |          |          |          |          |          |    |  |
| lui s0 1023  |    |    | IF | IF       | IF       | ID  | EX       | MEM<br>1 | MEM<br>2 | WB       |          |          |          |          |    |  |
| addi s0 s0 5 |    |    |    | ...      | ...      | IF  | ID       | EX       | MEM<br>1 | MEM<br>2 | WB       |          |          |          |    |  |
| ori s0 s0 1  |    |    |    |          | ...      | ... | IF       | ID       | EX       | MEM<br>1 | MEM<br>2 | WB       |          |          |    |  |
| srli s3 s0 4 |    |    |    |          |          |     |          | IF       | ID       | EX       | MEM<br>1 | MEM<br>2 | WB       |          |    |  |
| sw s0 8(sp)  |    |    |    |          |          |     |          |          | IF       | ID       | EX       | MEM<br>1 | MEM<br>2 | WB       |    |  |
| lw s1 8(sp)  |    |    |    |          |          |     |          |          |          | IF       | ID       | EX       | MEM<br>1 | MEM<br>2 | WB |  |

 s1 that is loaded at the Mem stage has to be used as an argument for the addi s2.

 We stall for 2 clock ticks.

 s0 that is loaded needs to be forwarded to addi s0 s0 15 to have the correct input value to perform the addi on.

 s0 that is loaded needs to be forwarded to ori s2 s0 1 to have the correct input value to perform the ori on.

 We tiebreak s0 with the most recent result. It can be from the previous (ex/ex), the penultimate (mem1/ex) or the second to last (mem2/ex).

 Ex 3 instructions ago: Hazard because s0 is decoded before the WB of s0. A solution is to stall for 1 clock tick.

Ex 4 instructions ago: For lui to sw: No hazard with reg file read bypass. The write back of s0 of lui will have ended before the read of s0 of srli. *SUBCOMPONENT(registers, TYPE(RegisterFile<XLEN, true>));*

 Store load hazard would happen if the mem2 stage of store will not have ended before the mem1 stage of load would begin. However, « we assume that these accesses will happen in order, e.g. a store followed by a load at the same address will not introduce a hazard. »

## Part 3 : Analysis

### QUESTION 1:

#### 6S\_BETTER.S:

The longer pipeline (6 stages) allows simpler stages, that reduces the clock cycle. This is useful for independent instructions that can be performed without needing to wait for the result of the previous instruction.

Examples of independent instructions in 4s\_better.s :

```
addi x1, x0, 1
addi x2, x0, 2
```

#### 4S\_BETTER.S

On the other hand, when the instructions depend on each other, and no forward is possible, the pipeline is forced to stall. My strategy to defect the 6-stage is to put them in stalling situations where it has to stall for a bigger number of clock cycles because of its bigger pipeline will penalize it:

```
#Data hazard: MemEx stalls for 1cycle in 4s, and for 2 cycles in 6s
lw x5, 0(x10)
add x6, x5, x7
```

This slows the 6s by  $2 \times 0.75 = 1.5$  which is more than 1 (of the 4s)

```
#Control hazards
#When jumping, 6s stalls for 3, 4s stalls for 2
addi s0, x0, 1
addi s1, x0, 3
jal addReg
addi s1, x0, 7
addi s1, x0, 7
addi s1, x0, 7
```

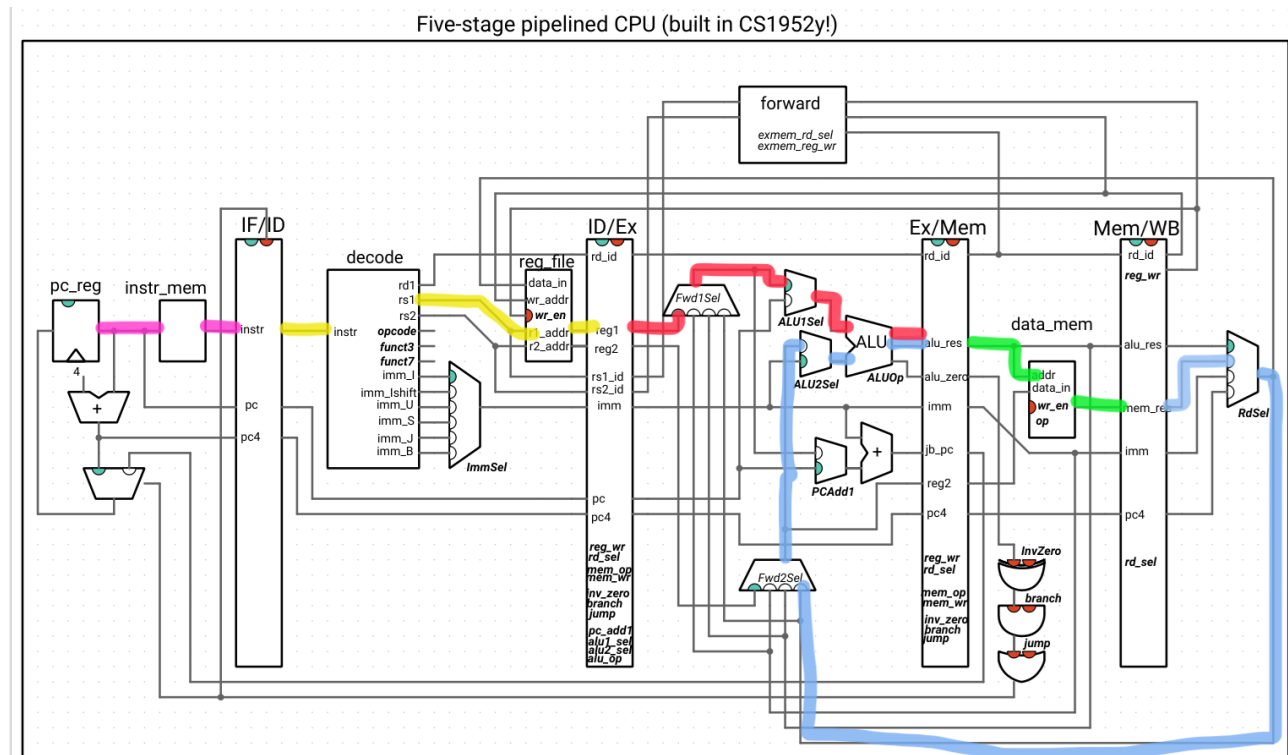
```
addReg: addi s2, x0, 1
addi s3, x0, 1
```

And this slows the 6s by  $3 \times 0.75 = 2.25$  which is more than 2 (of the 4s).

To maximize dependencies I did a large number of:

```
lw x1, 0(x1)
lw x1, 0(x1)
lw x1, 0(x1)
lw x1, 0(x1)
```

## QUESTION 2:

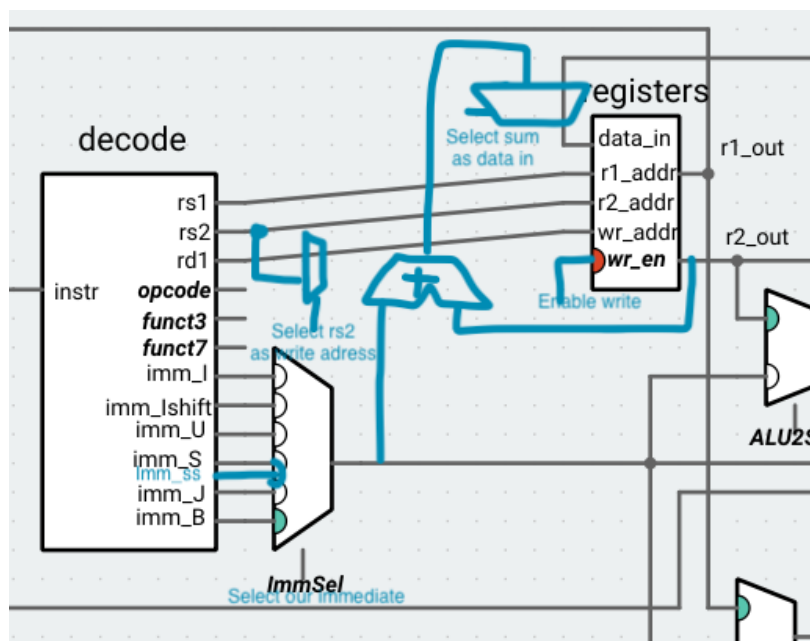
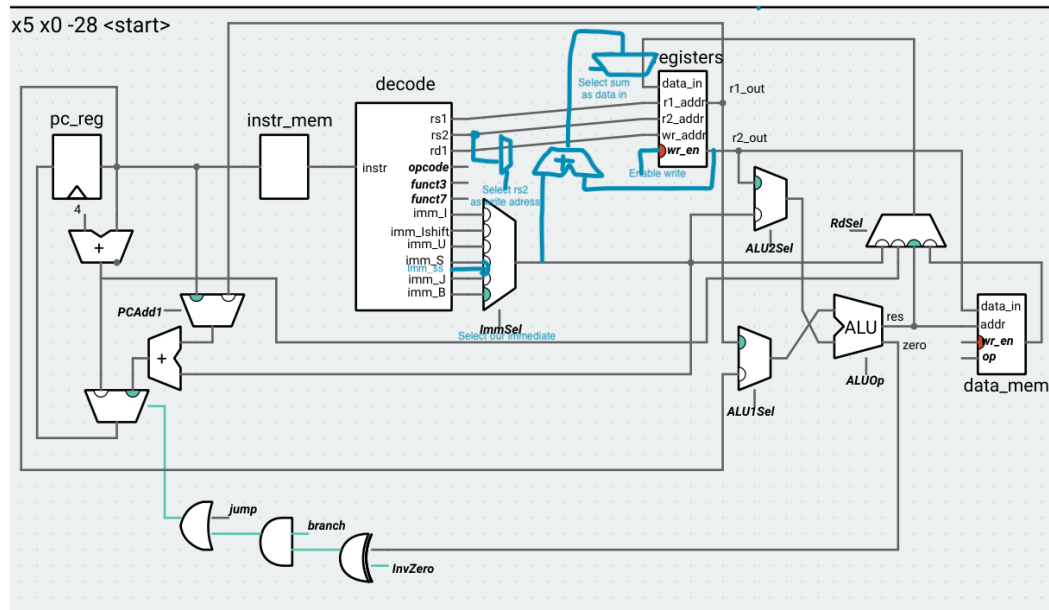


| Stage     | IF              | ID                   | EX                    | MEM             | WB                       |
|-----------|-----------------|----------------------|-----------------------|-----------------|--------------------------|
| Time (ps) | $30+250+20=300$ | $30+10+5+150+20=215$ | $30+25+25+150+20=250$ | $30+250+20=300$ | $30+25+25+25+150+20=275$ |

The minimum clock cycle time of the five stage CPU is then 300ps, corresponding to stages that use the memory.

### QUESTION 3:

Single-stage CPU (built from scratch in CS1952y!)

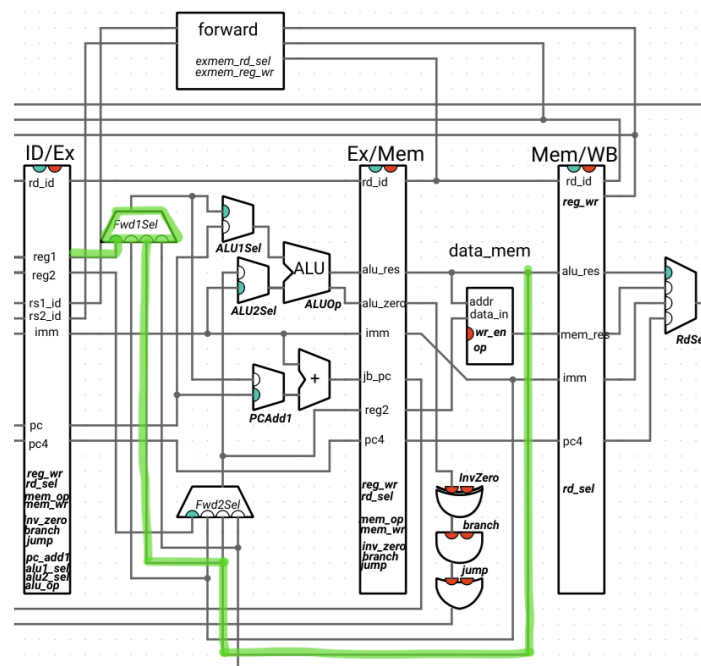


The following instructions would cause a hazard:

```
addi rs2, x0, 1
ss rs1, rs2, 2
```

Because rs1 is needed as an argument for ss. Without forward, rs1 wouldn't be updated until the WB, while rs1 is used in EX of ss.

We would need to forward the alu\_res in MEM of addi to be placed instead of reg1 in EX of ss. This wiring is already implemented (in green), so we would just need to update the logic of the forwarding unit.



## Part 4 : Analysis

### QUESTION 1:

This assignment made me realize the complexity of microprocessors. It is fascinating what has been done and all the smart techniques that have been developed to speed-up everything over the time, with pipelining, then forwarding...

I found very interesting to explore the trade-offs in the design, for the length of the pipeline for example.

### QUESTION 2:

It may be a good idea to divide it into two assignments in order to get feedback on the cpp implementation before using the processors that may have some incorrect parts. I found difficult understanding the logic.

### QUESTION 3:

In industry, are the circuits designed by hand like this, or to what extent it has been possible to automatize the process? To me it seems virtually impossible to explain our goals and trade-offs to a machine, other than designing and describing in detail in our VHDL.